Analyse : Structure Dictionnaire avec Algorithme de Martelli–Montanari

Ce document présente une analyse des performances de l'algorithme d'unification de Martelli–Montanari utilisé avec une structure de données de type dictionnaire pour organiser les littéraux par prédicat. Les expériences ont été réalisées en faisant varier la profondeur maximale des termes et l'arité maximale des prédicats générés aléatoirement.

```
--- Statistiques ---
Comparaisons : 125057
Unifications réussies : 11264
Échecs : 51187
Taille = 1000 littéraux | Temps = 0.301283 sec
RAM utilisée : 0.02 MB
Fin du traitement.

--- Statistiques ---
Comparaisons : 3124127
Unifications réussies : 335957
Échecs : 1226411
Taille = 5000 littéraux | Temps = 7.512308 sec
RAM utilisée : 0.00 MB
Fin du traitement.

--- Statistiques ---
Comparaisons : 12497058
Unifications réussies : 1302101
Échecs : 4942691
Taille = 10000 littéraux | Temps = 29.286494 sec
RAM utilisée : 0.09 MB
Fin du traitement.

--- Statistiques ---
Comparaisons : 312517679
Unifications réussies : 33430366
Échecs : 122834220
Taille = 50000 littéraux | Temps = 1688.304508 sec
RAM utilisée : -0.55 MB
```

Capture A : profondeurMax = 3, aritéMax = 3

```
Comparaisons : 124636
Unifications réussies : 13627
Échecs : 48311
Taille = 1000 littéraux | Temps = 1.269243 sec
RAM utilisée : 0.06 MB
Fin du traitement.


--- Statistiques ---
Comparaisons : 3123422
Unifications réussies : 347042
Échecs : 1215322
Taille = 5000 littéraux | Temps = 25.151371 sec
RAM utilisée : 0.12 MB
Fin du traitement.

--- Statistiques ---
Comparaisons : 12498595
Unifications réussies : 1441494
Échecs : 4803544
Taille = 10000 littéraux | Temps = 90.182848 sec
Unifications réussies : 1441494
Échecs : 4803544
Taille = 10000 littéraux | Temps = 90.182848 sec
RAM utilisée : 0.21 MB
Fin du traitement.
```

```
--- Statistiques ---
Comparaisons : 312523943
Unifications réussies : 35851657
Échecs : 120390057
Taille = 50000 littéraux | Temps = 7210.868150 sec
RAM utilisée : -36.52 MB
====== FIN BENCH ======
PS C:\Users\abogo\Documents\Initiation\M1-recherche>
```

Capture B : profondeurMax = 5, aritéMax = 5

Résultats des expériences :

Littéraux	Configuration	Comparaisons	Unifications réussies	Échecs	Temps (sec)
1000	(3,3)	125 057	11 264	51 187	0.30
5000	(3,3)	3 124 127	335 957	1 226 411	7.51
10000	(3,3)	12 497 058	1 302 101	4 942 691	29.29
50000	(3,3)	312 517 679	33 430 366	122 834 220	1688
1000	(5,5)	124 636	13 627	48 311	1.27
5000	(5,5)	3 123 422	347 042	1 215 322	25.15

10000	(5,5)	12 498 595	1 441 494	4 803 544	90.18
50000	(5,5)	312 523 943	35 851 657	120 390 057	7210

Analyse des résultats

1. Nombre de comparaisons :
Les comparaisons restent presque identiques entre les deux configurations. Cela s'explique par le fait que la structure dictionnaire regroupe les littéraux par prédicat. Les comparaisons sont donc principalement déterminées par le nombre total de littéraux et la distribution des prédicats, et non par la complexité interne des termes.

2. Impact sur le temps d'exécution :
Lorsque la profondeur et l'arité augmentent, le temps d'exécution augmente fortement. Par exemple, pour 10 000 littéraux, le temps passe d'environ 29 secondes (configuration 3,3) à environ 90 secondes (configuration 5,5). Pour 50 000 littéraux, le temps passe de 1688 secondes à plus de 7200 secondes. Cela montre que le coût principal provient de l'algorithme d'unification.

3. Complexité de l'unification :
Une profondeur plus grande produit des termes plus imbriqués. L'algorithme doit alors décomposer davantage de structures et générer plus d'équations. De même, une arité plus élevée augmente le nombre d'arguments à unifier, ce qui augmente le nombre d'équations à résoudre.

4. Taux d'unifications réussies :
Les résultats montrent une légère augmentation du nombre d'unifications réussies lorsque la profondeur et l'arité augmentent. Cela s'explique par la présence de plus de variables dans les termes complexes, ce qui rend les substitutions possibles plus fréquentes.

Conclusion

Les expérimentations montrent que l'utilisation d'une structure **dictionnaire** permet de réduire les comparaisons inutiles en regroupant les littéraux par prédicat. Contrairement à la structure de donnée **liste**, qui compare toutes les paires possibles, l'indexation limite les comparaisons aux littéraux partageant le même prédicat. Cela diminue le nombre d'appels à l'algorithme d'unification de Martelli-Montanari. Cependant, la complexité des termes (profondeur et arité) influence fortement le temps d'exécution de l'algorithme de Martelli–Montanari. Ainsi, même si la structure de données améliore la phase de filtrage, le coût principal reste celui de l'unification elle-même.